Day 6

Linear search:

It checks each element of a list one by one until the desired element is found or the list ends.

Input: *arr[] = [10, 50, 30, 70, 80, 20, 90, 40], x = 30*
Output : *2*
Explanation*: For array [10, 50, 30, 70, 80, 20, 90, 40], the element to be searched is 30 and it is at index 2. So, the output is 2.*

```python
def linear_search(arr, x):
    for i in range(len(arr)):
        if arr[i] == x:
            return i
    return -1

arr = [10, 23, 45, 70, 11, 15]
x = 70
res = linear_search(arr, x)

if res != -1:
    print("Element found at index:", res)
else:
    print("Element not found in the array")
```

Output

```
Element found at index: 3
```

Binary search:

It is an efficient searching algorithm used to find an element in a **sorted array** by repeatedly dividing the search interval in half. It reduces the time complexity to O(log N), making it much ***faster than linear search***.

Working -

1. Find the middle element of the array.
2. Compare the middle element with the target key. If equal: return index.
3. If the key is smaller: search the left half.
4. If the key is larger: search the right half.

5. Repeat until the element is found or the search space is empty.

```python
def binary_search(arr, x):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = low + (high - low) // 2

        if arr[mid] < x:
            low = mid + 1
        elif arr[mid] > x:
            high = mid - 1
        else:
            return mid
    return -1

arr = [2, 3, 4, 10, 40]
x = 10
result = binary_search(arr, x)

if result != -1:
    print("Element is present at index", result)
else:
    print("Element is not present in array")
```

Output

```
Element is present at index 3
```

Conditions to apply binary search:

- The input list must be sorted.
- Access to any element of the data structure should take constant time.

Search-space reduction:

Search-space reduction techniques in Python shrink the number of elements or states an algorithm must inspect, lowering time and space complexity.
Common reduction techniques -

1. Binary search
2. Ternary search
3. Jump search

Boundary conditions:

Boundary conditions represent the extreme limits or edge cases where an algorithm is most likely to fail. Handling boundary conditions properly means protecting your code from errors like IndexError, AttributeError (when encountering a None value), or infinite loops.

Data Structure	Critical Boundary Conditions to Check
Arrays / Lists	• Empty list (`len(arr) == 0`) • Single element (`len(arr) == 1`) • Out of bounds (`index >= len(arr)` or `index < 0`)
Linked Lists	• Empty list (`head is None`) • Single node (`head.next is None`) • Tail node pointer operations
Strings	• Empty string (`""`) • Single character • Case mismatch or special characters
Trees / Graphs	• Empty tree (`root is None`) • Leaf nodes (`node.left is None and node.right is None`)
Stacks / Queues	• Underflow (popping from an empty structure) • Overflow (for fixed-size buffers)

Recursion v/s Iteration:

Recursion	iteration
Uses repeated function calls	Uses control loops (for, while)
High memory usage	Low memory usage

Slower performance due to function call stack overhead	Faster performance as it avoids call stack generation
Used for: tree/graph traversal, backtracking, divide-and-conquer	Used for: linear sequences, array iteration, simple counting

Linear search:

```
#linear search

arr = (25, 34, 31, 45)
target = int(input())

def linear(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

print(linear(arr, target))
```

```
30
-1
```

Binary search:

```python
#binary search

arr = [10, 20, 30, 40, 50]
target = 40

def binary(arr, target):
  left, right = 0, len(arr) - 1
  while left <= right:
    mid = (left + right) // 2
    if arr[mid] == target:
      return mid
    elif arr[mid] < target:
      left = mid + 1
    else:
      right = mid - 1
  return -1
print(binary(arr, target))
```

```
3
```

First occurrence of target in sorted array:

```python
#First occurrence of a target in a sorted array

arr = [10, 20, 30, 30, 30, 40, 50]
target = 30

def first_occurrence(arr, target):
  left, right = 0, len(arr) - 1
  result = -1
  while left <= right:
    mid = (left + right) // 2
    if arr[mid] == target:
      result = mid
      right = mid - 1
    elif arr[mid] < target:
      left = mid + 1
    else:
      right = mid - 1
  return result

print(first_occurrence(arr, target))
```

```
2
```